VERIQTA

# Golang for Beginners:

# Your First 10 Scripts

## A Practical, Hands-On Guide to Writing Go Programs for DevOps, Cloud, and Infrastructure Engineers




- 10 Practical Go Scripts
- Real DevOps Use Cases
- Build CLI Tools
- Debug & Fix Errors
- Build, Test and Run

Linux | Docker | Kubernetes | AWS | CLI Tools | Automation

**Build Real-World Go Tools for Automation, Monitoring, and Operations**

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

# Why DevOps Engineers Learn Go

## What is Go?

- **Go (Golang)** is a modern, open source programming language created by Google.
- It is simple, fast, reliable, and built for building real-world tools and systems.

## Why Engineers Use Go

- ✅ Build fast and lightweight tools.
- ✅ Automate infrastructure tasks.
- ✅ Write reliable, production-ready programs.
- ✅ Easy to deploy (single binary).
- ✅ Widely used in cloud-native tools.

## Where Do You See Go?

| Docker | Kubernetes | Terraform | Cloud-Native Tools |
|---|---|---|---|
| (written in Go) | (written in Go) | (built with Go) | (many built with Go) |

## Compiled vs Interpreted

- **Compiled (Go):** Your code is converted to a standalone executable (fast, efficient, easy to deploy).
- **Interpreted (e.g., Python, Bash):** Runs line by line with an interpreter (more flexible but usually slower).

## From Code to Executable (Go Journey)

1. Write Code (.go file) → 2. Compile (go build) → 3. Executable (binary file) → 4. Run (./mytool)

## Real-World Example

**A simple Go tool that checks if an application is healthy.**

```
package main
import (
    "fmt"
    "net/http"
)
func main() {
   resp, err := http.Get("http://localhost:8080/health")
   if err != nil || resp.StatusCode != 200 {
      fmt.Println("❌ Application is NOT healthy")
   }else {
      fmt.Println("✅ Application is healthy")
} }
```

## Key Takeaway

- ✅ Go helps DevOps engineers build automation, CLI tools, and reliable infrastructure utilities.
- ✅ It powers major DevOps tools like Docker, Kubernetes, and Terraform.
- ✅ You write code, compile it, and run a fast, standalone program.

# Install Go and Run Your First Program

## 1. Install Go



- Download Go from: https://go.dev/dl/
- Install for your OS (Linux, macOS, or Windows).

## 2. Verify the Installatiom



`go version`

Expected output (example):

```
go version go1.22.5 linux/amd64
```

## 3. Create a Project and Initialize a Module

```
mkdir go-hello-devops
cd go-hello-devops
go mod init github.com/veriqta/hello
```

- Creates a project folder.
- Initializes a Go module (go.mod) to manage dependencies.

## 4. Understand the Basics

- **package main** – the entry point of a runnable Go program.
- **import** – brings in code from the Go standard library (e.g., fmt).
- **func main()** – the starting function that runs when you execute the program.

## 5. Write Your First Program

```go
package main

import "fmt"

func main() {
    fmt.Println("Hello DevOps!")
}
```

## 6. Run vs Build

| go run . | go build |
|---|---|
| Compiles and runs the program in one step. | Compiles the program into an executable file. |
| Best for development and quick testing. | Best for deployment and sharing. |
| Example: `go run .` | Example: `go build -o hello` `./hello` |

## 7. Troubleshooting Common Errors

| Error | Possible Cause | How to Fix |
|---|---|---|
| go: command not found | Go is not installed or not added to your system PATH. | Install Go from go.dev. Add Go to your PATH and restart your terminal. |
| go: go.mod file not found / outside a module | You are not inside a Go module (no go.mod). | Run: go mod init <module-name> Then run your command again. |

# Script 01: System Information Reporter

## What You Will Build



A simple Go command-line tool that identifies the environment where it runs (e.g., OS, architecture, and hostname).

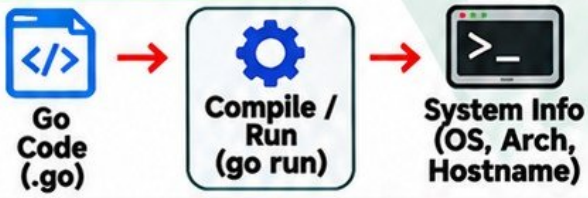
## Key Go Concepts Used

- **Variables:** Store data (e.g., os, arch, host).
- **Strings:** Text data, e.g., "linux".
- **Functions:** Reusable blocks of code.
- **runtime package:** Gets OS and system info.
- **os package:** Interacts with the operating system (e.g., hostname).

## Why It Matters

- Helps you quickly confirm which environment you are in.
- Useful when troubleshooting servers, containers, and deployments.
- Avoids running the wrong commands on the wrong system.

## How It Works




## The Go Program

```go
package main

import (
    "fmt"
    "os"
    "runtime"
)
func main() {
    osType := runtime.GOOS
    arch := runtime.GOARCH
    hostname, err := os.Hostname()
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println("System Information")
    fmt.Println("OS:", osType)
    fmt.Println("Architecture:", arch)
    fmt.Println("Hostname:", hostname)
}
```

## Example Output

```
$ go run main.go
System Information
OS: linux
Architecture: amd64
Hostname: app-server-01
```

## Practice

1. Run the program on your local machine:

   `go run main.go`

2. Run the same program inside a Linux container and compare the output:

   `docker run --rm golang:1.22 go run main.go`

## Key Takeaway

- ✅ You used runtime.GOOS and runtime.GOARCH to get system details.
- ✅ You used os.Hostname() to get the machine name with error handling.
- ✅ This kind of tool is a building block for health checks, diagnostics, and automation in real DevOps work.

# Script 02: File and Directory Inspector

## What You Will Build



A Go command-line tool that checks whether a path exists and tells you if it is a file or a directory.

## Key Go Concepts Used

- **os.Args:** Read command-line inputs.
- **os.Stat:** Get information about a path (file or directory).
- **IsDir():** Check if the path is a directory.
- **Error handling:** Manage missing paths and permission issues.

## Why It Matters

- Automation must verify required files and directories before running.
- Helps avoid failures in scripts, configurations, and deployments.
- Commonly used to check config files, log directories, and data folders.

## How It Works




## The Go Program (main.go)

```go
package main

import (
    "fmt"
    "os"
)
func main() {
    if len(os.Args) < 2 {
        fmt.Println("Usage: go run main.go <path>")
        return
    }
    path := os.Args[1]
    info, err := os.Stat(path)
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    if info.IsDir() {
        fmt.Println("Directory:", path)
    } else {
        fmt.Println("File:", path)
    }
}
```

## Example Output

```
$ go run main.go /etc/hosts
File: /etc/hosts

$ go run main.go /var/log
Directory: /var/log

$ go run main.go /no-such-path
Error: stat /no-such-path: no such file or directory
```

## Common Errors and Fixes

| Error | Cause | Fix |
|---|---|---|
| no such file or directory | The path does not exist. | Check the path and try again. |
| permission denied | You don't have access to the path. | Use a path you can access or run with proper permissions. |

## Practice

1. Check a real file (e.g., /etc/hosts):
   `go run main.go /etc/hosts`
2. Check a directory (e.g., /var/log):
   `go run main.go /var/log`
3. Check a nonexistent path:
   `go run main.go /no-such-path`

## Key Takeaway

- ✅ Use os.Args to accept input from the terminal.
- ✅ Use os.Stat to check if a path exists.
- ✅ Use IsDir() to tell if it is a file or a directory, and always handle errors.

# Script 03: Configuration File Reader

## What You Will Build




A Go program that reads an application configuration from a JSON file, validates the required settings, and handles errors safely.

## Key Go Concepts Used

- **Structs:** Define a data structure for your configuration.
- **JSON field tags:** Map struct fields to JSON keys (e.g., `` `json:"port"` ``).
- **os.ReadFile:** Read the JSON file from disk.
- **json.Unmarshal:** Convert JSON data into a Go struct.

## Configuration Example (config.json)

```
{
  "app_name": "veriqta-api",
  "port": 8080
  "log_ evel": "info",
  "database_url": "postgres://db:5432",
  "api_key": "***redacted**"
}
```

## The Go Program (main.go)

```
package main
import (
    "encoding/json"
    "fmt"
    "os"
)
type Config struct {
    AppName     string `json:"app_name"`
    Port        int    `json:"port"`
    LogLevel    string `json:"log_level"`
    DatabaseURL string `json:"database_url"`
    APIKey      string `json:"api_key"`
}
func main() {
    data, err := os.ReadFile("config.json")
    if err != nil {
        fmt.Println("Failed to read file:", err)
        return
    }
    var cfg Config
    if err := json.Unmarshal(data, &cfg); err != nil {
        fmt.Println("Invalid JSON:", err)
    }
    // Validate required settings
    if cfg.AppName ==  "" || cfg.Port == 0 {
        fmt.Println("Missing required settings:
        app_name and port are required")
    }
    fmt.Printf("App: %s | Port: %d | Log: %s\n",
        cfg.AppName, cfg.Port, cfg.LogLevel)
```

## Valid JSON vs Valid Configuration

| | |
|---|---|
| **Valid JSON** | The file is well-formed JSON (correct syntax). |
| **Valid Configuration** | The JSON has all the required settings for your application (e.g., app_name and port). |

## Detect Missing Settings

**Correct (required fields present)**

```
{"app_name": "api", "port": 8080 }
```

**Missing setting (port not provided)**

```
{"app_name": "api" }
```

## Why It Matters

- Applications often depend on config files (e.g., ports, database, log level).
- Validating the confr prevents startup failures and runtime errors.
- Common in Docker, Kubernetes, and CI/CD deployments.

## Security Reminder

- Never print passwords, API tokens, or other sensitive values.
- Keep secrets in environment variables or a secrets manager (e.g., AWS Secrets Manager, Kubernetes Secret).

## Practice

1. Create a config.json file (like the example).
2. Run your program: `go run main.go`
3. Remove the port and see the error message.

# Script 04: Application Log Analyzer

## What You Will Build




A Go tool that reads an application log file, counts log levels (ERROR, WARN, INFO), and gives a summary.

## Why It Matters

- Helps quickly spot issues in application and system logs.
- Useful for troubleshooting outages and failures.
- Common in DevOps and SRE tasks (e.g., log analysis, health checks).

## How It Works




## Sample Log File (app.log)

```
2026-10-31 10:00:01  INFO   Starting service
2026-10-31 10:00:05  WARN   High memory usage
2026-10-31 10:00:10  ERROR  Failed to connect to DB
2026-10-31 10:00:15  INFO   Retrying connection
2026-10-31 10:00:20  ERROR  DB connection timeout
```

## Example Output

```
$ go run main.go app.log
Log Summary
INFO :  2
WARN :  1
ERROR:  2
```

## Key Go Concepts Used

- **Loops:** Process each log line.
- **Conditions:** Check for log levels.
- **bufio.Scanner:** Read a file line by line (efficient and memory-friendly).
- **String matching:** Find text like ERROR, WARN, and INFO.

## The Go Program (main.go)

```go
package main
import (
    "bufio"
    "fmt"
    "os"
    "strings"
)
func main() {
    if len(os.Args) < 2 {
        fmt.Println("Usage: go run main.go <logfile>")
        return
    }
    file, err := os.Open(os.Args[1])
    if err != nil {
        fmt.Println("Error opening file:", err)
        return
    }
    defer file.Close()

    scanner := bufio.NewScanner(file)
    var info, warn, errCount int
    for scanner.Scan() {
        line := scanner.Text()
        if strings.Contains(line, "ERROR") {
            errCount++
        } else if strings.Contains(line, "WARN") {
            warn++
        } else if strings.Contains(line, "INFO") {
            info++
        }
    }
    if scanErr := scanner.Err(); scanErr != nil {
        fmt.Println("Read error:", scanErr)
        return
    }
    fmt.Println("Log Summary")
    fmt.Printf("INFO : %d\n", info)
    fmt.Printf("WARN : %d\n", warn)
    fmt.Printf("ERROR: %d\n", errCount)
}
```

## Common Errors and Fixes

| Error | Cause | Fix |
|---|---|---|
| open ...: permission denied | No read access to the file. | Use a readable file or update permissions (e.g., chmod +r). |
| no such file or directory | The log file does not exist. | Check the path and file name. |

## Key Takeaway

- ✅ Use bufio.Scanner to read large logs efficiently.
- ✅ Log analysis relies on consistent log formatting (e.g., the use of ERROR, WARN, INFO).
- ✅ Always handle file and scan errors to avoid silent failures.

## Practice

1. Create a sample log file (app.log) with INFO, WARN, and ERROR lines.
2. Run the program: `go run main.go app.log`
3. Modify the log file and confirm the counts change.

# Script 05: Disk Space Monitor

## What You Will Build



A Go tool that checks disk usage on Linux, shows total, used, and available space, and warns when usage exceeds a threshold.

## Key Go Concepts Used

- **Functions:** Reusable blocks of code.
- **Numeric calculations:** Compute sizes and usage percentage.
- **syscall.Statfs:** Get filesystem capacity (Linux).
- **Thresholds:** Trigger a warning when usage is high.

## Why It Matters

- Prevents application failures caused by a full disk.
- Avoids issues with logs, databases, and deployments.
- Helps you monitor storage usage proactively on servers and containers.

## How It Works



Get Disk Stats (Statfs) → Calculate Usage (Total, Used, Available) → Compare with Threshold and Warn

## Disk Space Terms

| | |
|---|---|
| Total | The total size of the filesystem. |
| Used | The space currently in use. |
| Available | The space available to unprivileged (non-root) processes. |
| Free | Total free space (may be higher than Available). |
| Usage % | (Used ÷ Total) × 100. |

## The Go Program (main.go)

```go
package main
import (
    "fmt"
    "os"
    "strconv"
    "syscall"
)
func main() {
    path := "/"                  // filesystem to check
    threshold := 80.0            // default 80%
    if len(os.Args) > 1 {
        if t, err := strconv.ParseFloat(os.Args[1], 64);
            err == nil {
            threshold = t
        }
    }
    var stat syscall.Statfs_t
    if err := syscall.Statfs(path, &stat); err != nil {
        fmt.Println("Error:", err)
        return
    }
    total := float64(stat.Blocks) * float64(stat.Bsize)
    free  := float64(stat.Bfree) * float64(stat.Bsize)
    avail := float64(stat.Bavail) * float64(stat.Bsize)
    used  := total - free
    usage := used / total * 100

    fmt.Printf("Path: %s\n", path)
    fmt.Printf("Total: %.2f GB\n", total/1e9)
    fmt.Printf("Used:  %.2f GB(%.1%)\n", used/1e9, usage)
    fmt.Printf("Available: %.2f GB\n", avail/1e9)
    if usage >= threshold {
        fmt.Printf("WARNING: Disk usage is %.1f% (>= %.1f%)\n",
            usage, threshold)
    } else {
        fmt.Println("Disk usage is within the limit.")
    }
}
```

## Example Output

```
$ go run main.go 80
Path: /
Total: 49.99 GB
Used:  42.31 GB (84.6%)
Available: 6.20 GB
WARNING: Disk usage is 84.6% (>= 80.0%)
```

## Common Issues and Fixes

| Issue | Cause | Fix |
|---|---|---|
| permission denied | No access to the filesystem. | Run with appropriate permissions (e.g., sudo). |
| invalid argument | Non-numeric threshold value. | Use a valid number (e.g., 80). |
| not supported (OS) | syscall.Statfs is Linux-specific. | Run on a Linux system (or use OS-specific logic). |

## Practice

1. Run the tool against your local filesystem: `go run main.go`
2. Change the threshold to 90% and observe the output: `go run main.go 90`

## Key Takeaway

- ✅ Use syscall.Statfs to read filesystem usage on Linux.
- ✅ Calculate total, used, and available space correctly.
- ✅ Set a threshold and warn when storage is running low.
- ✅ Monitoring disk space prevents outages and data loss.

# Script 06: HTTP Health Checker

## What You Will Build




A Go tool that sends an HTTP GET request to a web application's health endpoint, checks the response, and reports whether the application is healthy.

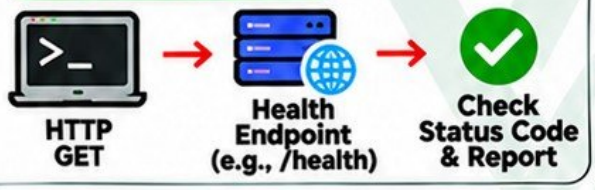
## Key Go Concepts Used

- **net/http:** Send HTTP requests and receive responses.
- **URLs:** The address of the health endpoint.
- **Status codes:** e.g., 200 (OK), 404 (Not Found), 500 (Server Error).
- **Timeout:** Prevents the request from hanging indefinitely.
- **Errors:** Handle network and request failures.

## Why It Matters

- Quickly confirms if an application is responding.
- Useful in automation, monitoring, and deployment checks.
- Helps detect outages and degraded services early.

## How It Works




## HTTP Status Code Quick Guide

| Code | Meaning |
|---|---|
| 200 | OK – the endpoint responded successfully. |
| 404 | Not Found – the endpoint does not exist. |
| 500 | Server Error – the application had an error. |
| Others | Other responses (e.g., 301, 403, 503) indicate specific conditions to investigate. |

## Example Output

```
$ go run main.go http://localhost:8080/health
Healthy: HTTP 200 OK
$ go run main.go http://localhost:8080/unknown
Unhealthy: HTTP 404
$ go run main.go http://localhost:9999/health
Network error: dial tcp 127.0.0.1:9999: connect:
connection refused
```

## Why 200 Alone Is Not Enough

- HTTP 200 only means the endpoint responded.
- The application may still have internal failures (e.g., database, queue, or dependencies down).
- A better health check should also verify critical dependencies (e.g., database, cache, external APIs).

## The Go Program (main.go)

```go
package main
import (
    "fmt"
    "net/http"
    "os"
    "time"
)
func main() {
    url := "http://localhost:8080/health"
    if len(os.Args) > 1 {
        url = os.Args[1] // allow custom URL
    }
    client := &http.Client{
        Timeout: 5 * time.Second,
    }
    resp, err := client.Get(url)
    if err != nil {
        fmt.Printf("Network error: %v\n", err)
        return
    }
    defer resp.Body.Close()

    if resp.StatusCode == http.StatusOK{
        fmt.Println("Healthy: HTTP 200 OK")
    } else {
        fmt.Printf("Unhealthy: HTTP %d\n", resp.StatusCode)
    }
}
```

## HTTP Error vs Network Failure

| Type | Example | Meaning |
|---|---|---|
| HTTP Error (response) | 404, 500, 503 | The server responded, but with an error status code.<br>The application is reachable but not working as expected. |
| Network Failure | connection refused, timeout, no route to host | No response from the server.<br>The service may be down, not listening, or blocked by a firewall. |

## Practice

1. Test a working endpoint: `go run main.go http://localhost:8080/health`
2. Test a nonexistent path: `go run main.go http://localhost:8080/unknown`
3. Test an unreachable port: `go run main.go http://localhost:9999/health`

# Script 07: TCP Port Connectivity Checker

## What You Will Build



A Go tool that tests whether a remote TCP service accepts connections and reports the result with a clear message.

## Key Go Concepts Used

- **net.DialTimeout:** Open a TCP connection with a timeout.
- **Hostnames:** Domain name or IP address (e.g., db.veriqta.com, 10.0.0.5).
- **Ports:** The service port (e.g., 80, 443, 5432).
- **Timeouts:** Limit how long to wait for a response.
- **Errors:** Handle and report connection failures.

## Why It Matters

- Quickly verifies if a service is reachable (e.g., database, API, message broker).
- Helps troubleshoot connectivity issues in Linux, Kubernetes, and cloud environments.
- Useful during deployments, incident response, and system checks.

## How It Works




## The Go Program (main.go)

```go
package main
import (
    "fmt"
    "net"
    "os"
    "time"
)
func main() {
    if len(os.Args) != 3 {
        fmt.Println("Usage: go run main.go <host> <port>")
        return
    }
    host := os.Args[1]
    port := os.Args[2]
    address := net.JoinHostPort(host, port)
    timeout := 5 * time.Second
    conn, err := net.DialTimeout("tcp", address, timeout)
    if err != nil {
        fmt.Printf("Connection failed: %v\n", err)
        return
    }
    defer conn.Close()
    fmt.Printf("Connected to %s (TCP) successfully\n",
        address)
}
```

## Key TCP Outcomes

| Result | Meaning |
|---|---|
| Connected | The remote service is reachable and accepts TCP connections. |
| Connection Refused | The host is reachable, but no service is listening on the port. |
| Connection Timed Out | No response within the timeout (possible network block, firewall, or service is down). |

## Connection Refused vs Timed Out

| Error | What It Means | Common Causes |
|---|---|---|
| Connection Refused | The host is reachable, but the port is closed (no service is listening). | Service not running.<br>Wrong port. |
| Connection Timed Out | No response from the host within the timeout. | Network block (firewall/ security group).<br>Host is down or unreachable. |

## Network Connectivity vs Application Health

| | |
|---|---|
| Network Connectivity | Confirms the port is reachable (e.g., you can connect to the service). |
| Application Health | Confirms the application is working correctly (e.g., by checking a /health endpoint or service status). |

## Practice

1. Test a listening port (e.g., a local web server):
   `go run main.go 127.0.0.1 8080`
2. Test a closed port (no service running):
   `go run main.go 127.0.0.1 9999`
3. Try a remote service (e.g., a database port on your lab):
   `go run main.go localhost 5432`

# Script 08: Environment Variable Validator

## What You Will Build




A Go tool that checks whether required environment variables are set and valid before an application starts.

## Key Go Concepts Used

- **Slices:** A list of required variables (e.g., []string{"DB_HOST", "DB_USER"}).
- **Loops:** Check each variable in the list.
- **os.LookupEnv:** Read an environment variable and check if it is set.
- **Conditions:** Distinguish missing vs empty variables.

## Why It Matters

- Prevents application startup failures due to missing configuration.
- Helps ensure a consistent and reliable deployment.
- Common in containerized and cloud-native applications (e.g., Docker, Kubernetes).

## How It Works




## Variable Status

| Status | Meaning |
|---|---|
| Set (Valid) | The variable exists and has a non-empty value. |
| Missing | The variable is not set (not found). |
| Empty | The variable exists but its value is an empty string (""). |

## The Go Program (main.go)

```go
package main
import (
    "fmt"
    "os"
)
func main() {
    // required environment variables
    required := []string{
        "DB_HOST",
        "DB_USER",
        "DB_PASSWORD",
        "APP_ENV",
    }
    var missing []string
    var empty []string
    for _, name := range required {
        val, ok := os.LookupEnv(name)
        if !ok {
            missing = append(missing, name)
        } else if val == "" {
            empty = append(empty, name)
        }
    }
    if len(missing) > 0 || len(empty) > 0 {
        fmt.Println("Configuration validation failed")
        if len(missing) > 0 {
            fmt.Printf("Missing variables: %v\n", missing)
        }
        if len(empty) > 0 {
            fmt.Printf("Empty variables: %v\n", empty)
        }
        os.Exit(1)
    }
    fmt.Println("All required environment variables are set.")
}
```

## Important Notes

- Environment variables are useful for configuration (e.g., database, API, region, mode).
- They are NOT automatically secure storage for secrets.
- On the same machine, other processes or users may be able to read environment variables (e.g., via ps, /proc, or application logs).
- Use a secrets manager (e.g., AWS Secrets Manager, HashiCorp Vault, Kubernetes Secrets) for sensitive values in production.

## Practice

1. Set sample variables:
   `export DB_HOST=localhost` `export DB_USER=admin`
2. Run the validator:
   `go run main.go`
3. Unset a variable and run again:
   `unset DB_HOST` && `go run main.go`
4. Set an empty value and test:
   `export APP_ENV=""` && `go run main.go`

# Script 09: Concurrent Endpoint Monitor

## What You Will Build




A Go program that checks multiple application endpoints concurrently and reports the health status of each.

## Key Go Concepts Used

- **Goroutines:** Run multiple tasks at the same time.
- **sync.WaitGroup:** Wait for all goroutines to finish.
- **HTTP client & timeouts:** Make safe requests to endpoints.
- **Mutex (sync.Mutex):** Protect shared results (e.g., a map) from concurrent writes.

## Why It Matters

- Checks many services faster (e.g., API, DB, auth, etc.).
- Gives a real-time view of system health.
- Useful for monitoring, incident response, and automated checks.

## How It Works




## Sequential vs Concurrent

| Method | How It Works | Time (Example) |
|---|---|---|
| Sequential | Checks endpoints one after another. | ~ 6 seconds (3 × 2s each) |
| Concurrent | Checks all endpoints at the same time (goroutines). | ~ 2 seconds (≈ longest single request) |

## Common Results

| Result | Meaning |
|---|---|
| 200 – 299 | Endpoint is healthy (success). |
| 400 – 499 | Client error (e.g., bad request, unauthorized). |
| 500 – 599 | Server error (application or service issue). |
| Error (timeout, connection refused) | The endpoint could not be reached (network or service issue). |

## The Go Program (main.go)

```go
package main
import (
    "fmt"
    "net/http"
    "sync"
    "time"
)
type Result struct {
    URL    string
    Status int
    OK     bool
    Error  string
}
func checkEndpoint(client *http.Client, url string,
    wg *sync.WaitGroup, mu *sync.Mutex, results map[string]Result) {
    defer wg.Done()
    resp, err := client.Get(url)
    r := Result{URL: url}
    if err != nil {
        r.OK = false
        r.Error = err.Error()
    } else {
        r.Status = resp.StatusCode
        r.OK = resp.StatusCode >= 200 && resp.StatusCode < 300
        resp.Body.Close()
    }
    mu.Lock()
    results[url] = r
    mu.Unlock()
}
func main() {
    endpoints := []string{
        "http://localhost:8080/health",
        "http://localhost:8080/ready",
        "http://localhost:8080/live",
    client := &http.Client{Timeout: 5 * time.Second}
    results := make(map[string]Result)
    var wg sync.WaitGroup
    var mu sync.Mutex
    for _, url := range endpoints {
        wg.Add(1)
        go checkEndpoint(client, url, &wg, &mu, results)
    wg.Wait()
    for _, r := range results {
        if r.OK {
            fmt.Printf("✔ %s - %d (OK)\n", r.URL, r.Status
        } else {
            fmt.Printf("✖ %s - Failed: %s\n", r.URL, r.Error)
}
```

## Key Takeaway

- Use goroutines + WaitGroup for concurrent checks.
- Protect shared data (e.g., a map) with a mutex.
- Concurrency reduces total check time, but too many concurrent requests can overload the service.
- A healthy HTTP response (e.g., 200) is useful, but combine it with deeper checks (e.g., dependencies, database, queue).

## Practice

1. Monitor three endpoints: `/health, /ready, and /live.`
2. Run the program: `go run main.go`
3. Observe the results and compare the total time with a sequential version.

# Script 10: Automated File Backup Tool

## What You Will Build



A Go tool that safely copies a file to a backup directory with a timestamp, preserves permissions, and handles errors.

## Key Go Concepts Used

- **io.Copy:** Efficiently copy file contents.
- **os.Create:** Create the backup file.
- **time:** Generate a timestamp for backup names.
- **os.Chmod:** Set file permissions.
- **defer:** Ensure files are closed (cleanup).

## Why It Matters

- Protects important configuration, logs, and data files.
- Helps you recover from mistakes (e.g., bad deployment or corrupt file).
- Useful in automation, maintenance scripts, and incident response.

## How It Works




## The Go Program (main.go)

```go
package main
import (
    "fmt"
    "io"
    "os"
    "path/filepath"
    "time"
)
func main() {
    if len(os.Args) != 3 {
        fmt.Println("Usage: go run main.go <source-file> <backup-dir>")
        os.Exit(1)
    }
    src := os.Args[1]
    dstDir := os.Args[2]
    // ensure backup directory exists
    if err := os.MkdirAll(dstDir, 0755); err != nil {
        fmt.Printf("Failed to create backup directory: %v\n", err)
        os.Exit(1)
    }
    // open source file
    srcFile, err := os.Open(src)
    if err != nil {
        fmt.Printf("Failed to open source file: %v\n", err)
        os.Exit(1)
    }
    defer srcFile.Close()
    // create timestamped backup file
    ts := time.Now().Format("20060102-150405")
    base := filepath.Base(src)
    backupPath := filepath.Join(dstDir, base+"."+ts+".bak")
    dstFile, err := os.OpenFile(backupPath, os.O_CREATE|os.O_EXCL|os.O_WRONLY, 0644)
    if err != nil {
        fmt.Printf("Failed to create backup file: %v\n", err)
        os.Exit(1)
    }
    defer dstFile.Close()
    // copy contents
    if _, err := io.Copy(dstFile, srcFile); err != nil {
        fmt.Printf("Copy failed: %v\n", err)
        os.Exit(1)
    }
    // preserve permissions
    if info, err := os.Stat(src); err == nil {
        os.Chmod(backupPath, info.Mode())
    }
    fmt.Printf("Backup created: %s\n", backupPath)
}
```

## Important Go APIs

| API | Purpose |
|---|---|
| os.Open | Open the source file for reading. |
| os.Create / os.OpenFile | Create the backup file (e.g., with O_EXCL to avoid overwrite). |
| io.Copy | Copy file contents from source to backup. |
| time.Now() | Generate a timestamp for unique backup names. |
| os.Chmod | Set file permissions (preserve original). |
| defer | Ensure files are closed, even on errors. |

## Backup vs Recoverable Backup

| | |
|---|---|
| Successful Copy | The file was copied without an error. It only confirms the copy operation completed. |
| Verified, Recoverable Backup | The backup is usable (not corrupted), can be restored, and has been tested (e.g., by reading or restoring the file). |

## Key Takeaway

- ✅ Create timestamped backups to avoid overwrites.
- ✅ Always handle errors and close files with defer.
- ✅ Preserve file permissions when possible.
- ✅ A successful copy ≠ a guaranteed, recoverable backup.
- ✅ Periodically test your backups by restoring a file.

## Practice

1. Create a test file: `echo "app config" > app.conf`
2. Run the backup tool: `go run main.go app.conf ./backups`
3. Verify the backup: `ls -l ./backups`
4. Compare the contents: `diff app.conf ./backups/app.conf.*.bak`

## Example Output

```
$ go run main.go app.conf ./backups
Backup created: ./backups/app.conf.20261031-142530.bak

$ ls -l ./backups
total 4
-rw-r--r-- 1 ann ann  11 Oct 31 14:25
app.conf.20261031-142530.bak
```

# Understand Go Errors Instead of Ignoring Them

## The Core Idea




In Go, many functions return two values: the result and an error. You must check the error and handle it instead of ignoring it.

## Key Go Concepts

- **value, err:** Common return pattern in Go.
- **if err != nil:** Check if an operation failed.
- **errors.Is:** Check for a specific, expected error (e.g., file not found).
- **fmt.Errorf("... %w", err):** Wrap an error with more context while keeping the original error.

## Why It Matters

- Tells you when something went wrong.
- Helps you handle expected problems gracefully (e.g., missing file).
- Adds useful context for troubleshooting (e.g., which file, which operation).
- Makes your tools reliable and production-ready.

## The Pattern: Value + Error

```go
data, err := os.ReadFile("app.conf")
if err != nil {
    // handle the error
} else {
    // use data
```

- Two returns: (data, err)
- Check if err != nil
- Handle the error or use the result.

## Understanding errors.Is

```go
info, err := os.Stat("app.conf")
if err != nil {
    if errors.Is(err, os.ErrNotExist) {
        fmt.Println("File not found")
    } else {
        fmt.Println("Stat error:", err)
    }
} else {
    fmt.Printf("Size: %d bytes\n", info.Size())
}
```

## Wrapping Errors for Better Context

```go
f, err := os.Open("app.conf")
if err != nil {
    return fmt.Errorf("open config file %s: %w",
        "app.conf", err)
}
```

%w wraps the original error, so you keep the details (e.g., file not found) while adding context (e.g., "open config file app.conf").

## Examples from Our Tools

| Tool | Example of Proper Error Handling |
|---|---|
| File Inspector | `info, err := os.Stat(path)`<br>`if err != nil {`<br>`  if errors.Is(err, os.ErrNotExist) {`<br>`  fmt.Println("Path does not exist")`<br>`  } else {`<br>`  fmt.Printf("Cannot inspect %s: %v\n", path, err)`<br>`}` |
| Backup Tool | `_, err := io.Copy(dst, src)`<br>`if err != nil {`<br>`  return fmt.Errorf("backup failed: %w", err)`<br>`}` |
| Health Checker | `resp, err := client.Get(url)`<br>`if err != nil {`<br>`  return fmt.Erroff("health check failed for %s: %w",`<br>`}` |

## Common Mistakes to Avoid

| | |
|---|---|
| **Ignoring errors** | `data, _ := os.ReadFile("app.conf")`<br>This hides failures and can lead to unexpected problems. |
| **Printing raw errors only** | `fmt.Println(err)`<br>Works, but often lacks context (e.g., which file or operation failed). |
| **Not handling expected errors** | Treating all errors the same (e.g., not checking for os.ErrNotExist) makes your tool less user-friendly. |

## Key Takeaway

- Always check and handle err != nil.
- Use errors.Is for expected conditions.
- Wrap errors with fmt.Errorf("... %w", err) to add context.
- Never ignore errors, even in small scripts.

## Practice

1. Modify the file inspector to print a clear message when a file does not exist using errors.Is.
2. Update the backup tool to wrap copy errors with context.
3. Add error handling in the health checker for network failures (e.g., timeout or connection refused).

# Turn Hardcoded Scripts into Command-Line Tools

## The Core Idea



Convert fixed, hardcoded scripts into flexible command-line tools that accept options (e.g., endpoint, timeout, file path, threshold).

## Key Go Concepts

- **flag:** Standard package for command-line flags.
- **Flags:** Named inputs (e.g., -url, -timeout, -file).
- **Input validation:** Check required and valid values.
- **Usage information:** Show help with -h or -help.
- **Security:** Avoid passing secrets on the command line (they can appear in logs or process listings).

## Why It Matters

- Hardcoded values limit reuse and require code changes.
- Flags make the tool flexible for different environments.
- Improves usability, automation, and integration (e.g., in CI/CD).

## How It Works




## Common Command-Line Flags

| Flag | Example | Purpose |
|---|---|---|
| -url | -url https://app/veriqta | The endpoint to check (HTTP health check). |
| -timeout | -timeout 10 | Request timeout in seconds. |
| -file | -file /etc/app.conf | Path to a file (e.g., backup or inspect). |
| -threshold | -threshold 80 | Disk usage threshold (e.g., 80%). |

## Usage and Help Output

```
$ go run main.go -h
Usage of main.go:
  -url string
    Endpoint to check (e.g., https://app.veriqta.com/health)
  -timeout int
    Request timeout in seconds (default 5)
```

## The Go Program (main.go)

```go
package main
import (
    "flag"
    "fmt"
    "net/http"
    "time"
)
func main() {
    url := flag.String("url", "", "Endpoint to check
    (e.g., https://app.veriqta.com/health)")
    timeout := flag.Int("timeout", 5, "Request timeout
    in seconds")
    flag.Parse()
    // validate input
    if *url == "" {
        fmt.Println("Error: -url is required.")
        fmt.Println("Usage: go run main.go -url <endpoint> \
        -timeout <seconds>")
        return
    }
    client := &http.Client{
        Timeout: time.Duration(*timeout) * time.Second,
    }
    resp, err := client.Get(*url)
    if err != nil {
        fmt.Printf("Request failed: %v\n", err)
        return
    }
    defer resp.Body.Close()
    fmt.Printf("Status: %d\n", resp.StatusCode)
}
```

## Important Notes

- Validate user input (e.g., required, correct format).
- Use -h or -help to show clear usage instructions.
- Do not pass secrets (passwords, tokens) via command-line flags, as they can appear in process listings (ps, top) and logs.
- Use environment variables or a secure config file for sensitive values.

## Practice

1. Run the health checker with a custom endpoint:
   `go run main.go -url https://app.veriqta.com/health -timeout 10`
2. Try without the required flag and observe the error:
   `go run main.go`
3. Add input validation (e.g., ensure the timeout > 0).

## Key Takeaway

- Use the flag package to make your tools configurable.
- Replace hardcoded values with command-line flags.
- Validate inputs and provide helpful usage information.
- Keep secrets out of command-line arguments.

# Organize Your Go Projects Like an Engineer

## The Core Idea




Structure your Go code into a clean, maintainable project with modules, packages, and separate commands for each tool.

## Why It Matters

- Keeps your code organized and scalable.
- Allows code reuse across multiple tools (e.g., HTTP, file, logging helpers).
- Makes the project easier to test, maintain, and extend.

## How It Works

go.mod go.sum (Manage Dependencies) → cmd/ (Entry Points for Each Tool) → internal/ (Reusable Logic) → A Clean, Scalable Go Project

## Module Files (Example)

| File | Purpose |
|---|---|
| go.mod | Defines the module name and Go version, e.g., module github.com/veriqta/go-tools. |
| go.sum | Records the exact versions of dependencies for consistent builds. |

## Package and Imports (Example)

```go
// internal/httputil/health.go
package httputil

import (
    "fmt"
    "net/http"
)
// Exported function (reusable by other commands)
func CheckHealth(url string) error {
    resp, err := http.Get(url)
    if err != nil {
        return fmt.Errorf("health check failed: %w", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        return fmt.Errorf("unexpected status: %d", resp.StatusCode)
    }
    return nil
}
```

## Key Takeaway

- Use go.mod and go.sum to manage your project and dependencies.
- Put each tool in cmd/ and reusable code in internal/ or pkg/.
- Use exported functions (uppercase) for reusable logic.
- Run gofmt regularly for clean, consistent code.

## Key Go Concepts

- **go.mod:** Module file that defines your project name, Go version, and dependencies.
- **go.sum:** Lock file that records exact dependency versions for reproducible builds.
- **Package:** A collection of related Go code (files).
- **Exported function:** Starts with an uppercase letter (e.g., CheckHealth) and can be used by other packages.
- **gofmt:** Standard tool that formats Go code consistently.

## A Practical Project Structure

```
go-tools/
├── go.mod
├── go.sum
├── cmd/
│   ├── file-inspector/          # Script 01
│   ├── log-analyzer/            # Script 04
│   ├── disk-monitor/            # Script 05
│   ├── health-check/            # Script 06
│   ├── port-check/              # Script 07
│   ├── env-validator/           # Script 08
│   ├── endpoint-monitor/        # Script 09
│   └── backup-tool/             # Script 10
├── internal/
│   ├── fileutil/        # file helpers (copy, perms)
│   ├── httputil/        # HTTP client, health checks
│   ├── sysutil/         # disk, system checks
│   ├── logger/          # logging helpers
│   └── config/          # shared config and validation
├── pkg/
│   ├── errors/          # custom error helpers
│   └── types/           # shared types (e.g., Result)
└── README.md
```

## gofmt: Keep Code Consistent

```
$ gofmt -w .
```

- Formats all Go files in the current project.
- Ensures consistent style (indentation, spacing, imports).
- Helps keep the code clean and readable across all scripts.

## Project Structure for the 10 Scripts

| Script (Tool) | Command Directory |
|---|---|
| 01. File Inspector | cmd/file-inspector/ |
| 04. Log Analyzer | cmd/log-analyzer/ |
| 05. Disk Space Monitor | cmd/disk-monitor/ |
| 06. HTTP Health Checker | cmd/health-check/ |
| 07. TCP Port Checker | cmd/port-check/ |
| 08. Env Variable Validator | cmd/env-validator/ |
| 09. Concurrent Endpoint Monitor | cmd/endpoint-monitor/ |
| 10. Automated File Backup | cmd/backup-tool/ |

## Practice

1. Create a new project: `go mod init github.com/veriqta/go-tools`
2. Create the structure (cmd/, internal/, pkg/) and move one of your scripts (e.g., health checker) into it.
3. Run `gofmt -w .` and verify the project builds: `go build ./...`

# Test Your Go Scripts Before Trusting Them

## The Core Idea




Use Go's built-in testing package to verify that your scripts work as expected under different conditions.

## Key Go Concepts

- **testing package:** Built-in framework for tests.
- **Unit test:** Verifies a small, specific function.
- **Table-driven test:** Runs the same test logic with multiple input cases.
- **go test ./...:** Runs tests in the current module and all subpackages.
- **go test -race ./...:** Detects data races (concurrency issues).

## Why It Matters

- Catches errors early before deployment.
- Prevents automation mistakes (e.g., wrong thresholds, bad configs).
- Makes your scripts more reliable and easier to change safely.

## What Unit Tests Verify

- ✅ Functions return the expected result (e.g., configuration is valid or invalid).
- ✅ Handle both success and failure cases.
- ✅ Behave correctly with different inputs (e.g., missing or empty values).

## How It Works




## Table-Driven Test Example

```go
package config

import "testing"

type Config struct {
    Endpoint string
    Timeout  int
}
func ValidateConfig(c Config) error {
    if c.Endpoint == "" { return fmt.Errorf("endpoint required") }
    if c.Timeout <= 0  { return fmt.Errorf("timeout must be > 0") }
    return nil
}
func TestValidateConfig(t *testing.T) {
    tests := []struct {
        name    string
        cfg     Config
        wantErr bool
    }{
        {"valid config", Config("https://api.veriqta.com", 5}, false},
        {"missing endpoint", Config("", 5}, true},
        {"invalid timeout", Config("https://api.veriqta.com", 0}, true},
    }
    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            err := ValidateConfig(tc.cfg)
            if (err != nil) != tc.wantErr {
                t.Fatalf("got err=%v, wantErr=%v", err, tc.wantErr)
            }
        })
    }
}
```

## Go Test Commands

| Command | Purpose |
|---|---|
| go test | Run tests in the current package. |
| go test ./... | Run tests in the current module and all subpackages. |
| go test -v ./... | Run tests with detailed output (show each test). |
| go test -race ./... | Run tests and check for data races (concurrent access issues). |

## Example Test Output

```
$ go test -v ./...
=== RUN   TestValidateConfig
=== RUN   TestValidateConfig/valid_config
=== RUN   TestValidateConfig/missing_endpoint
=== RUN   TestValidateConfig/invalid_timeout
=== PASS: TestValidateConfig (0.00s)
    --- PASS: TestValidateConfig/valid_config (0.00s)
    --- PASS: TestValidateConfig/missing_endpoint (0.00s)
    --- PASS: TestValidateConfig/invalid_timeout (0.00s)
PASS
ok      github.com/veriqta/go-tools/config    0.015s
```

## Key Takeaway

- ✅ Use Go's testing package to verify your script logic.
- ✅ Write table-driven tests for multiple input cases.
- ✅ Run `go test ./...` regularly.
- ✅ Use `go test -race ./...` for concurrent code (e.g., the concurrent endpoint monitor).

## Why Tests Prevent Mistakes

- ✅ Catch logic errors (e.g., wrong thresholds or validation).
- ✅ Ensure changes do not break existing behavior.
- ✅ Build confidence to automate and run in production.
- ✅ Help you refactor and improve your code safely.

## Practice

1. Create a test file (e.g., config/config_test.go).
2. Add a table-driven test for your configuration validator.
3. Run the tests: `go test -v ./...` and `go test -race ./...`

# Debug Common Go Errors

## The Core Idea




Learn to read, understand, and fix common Go errors so you can solve real problems faster and confidently.

## Key Concepts

- **Error message:** Tells you what went wrong.
- **Cause:** The issue in your code or environment.
- **Fix:** The correct change to resolve it.
- **Verify:** Rerun the script to confirm it works.

## Why It Matters

- Errors are part of real development.
- Knowing common errors saves time.
- Helps you build reliable scripts and tools.
- Builds problem-solving skills for production work.

## The Debugging Process (Repeatable)

1. Read the error message
2. Identify the failing operation
3. Inspect the relevant code
4. Correct the cause (fix the code or config)
5. Verify the error is resolved

## Common Go Errors and How to Fix Them

| Error | What It Means | Example (Code / Message) | How to Fix It |
|---|---|---|---|
| undefined: | A name is not declared or accessible (e.g., variable, function). | `fmt.Println(name)`<br>`// error: undefined: name` | Declare the variable or use the correct name: `name := "veriqta"` |
| imported and not used: | An imported package is not used in the code. | `import "os"`<br>`// error: os imported and not used` | Remove the unused import or use it in the code (e.g., `os.Getenv("HOME")`) |
| assignment mismatch: | The number of returned values does not match the assignment. | `f, err := os.Open("app.conf")`<br>`name := os.Getenv("HOME")`<br>`// error: assignment mismatch:`<br>`// 2 variables but 1 value` | Match the variables to the returned values: `name, err := os.Getenv("HOME")` |
| panic: runtime error: | A runtime failure (e.g., nil pointer, index out of range). | `var p *string`<br>`fmt.Println(*p)`<br>`// panic: runtime error:`<br>`// invalid memory address` | Check for nil or validate values before use: `if p != nil { fmt.Println(*p) }` |
| permission denied: | The operating system rejected the operation (e.g., reading a restricted file). | `os.Open("/root/secret.txt")`<br>`// open /root/secret.txt:`<br>`// permission denied` | Use a file you have access to, run with appropriate permissions, or change file permissions (e.g., chmod). |
| connection refused: | The target TCP connection was rejected (no service listening on that port). | `http.Get("http://localhost:8080")`<br>`// dial tcp 127.0.0.1:8080:`<br>`// connect: connection refused` | Ensure the service is running, check the correct host and port, and confirm the port is listening (e.g., ss -ltn or netstat -ltn). |

## Key Takeaway

- [x] Read the error message carefully.
- [x] Map the error to its likely cause.
- [x] Fix the code or environment.
- [x] Run the script again to confirm the fix.
- [x] Build a personal list of common errors and solutions.

## Practice

1. Create small scripts that intentionally trigger each error type above.
2. Fix each script using the debugging process.
3. Verify the fixes by running the scripts successfully (e.g., `go run main.go`)

# Security Rules for Junior Go Engineers

## The Core Idea




Build safe, reliable Go tools by following simple security rules that prevent leaks, misuse, and unwanted access.

## Key Go Concepts

- **Credentials:** Secrets such as passwords, API keys, and tokens.
- **Environment variables:** Configuration values from the environment (e.g., os.Getenv).
- **File permissions:** Access control for files (e.g., 0600).
- **Timeouts:** Limit how long a network operation waits (e.g., context or http.Client timeout).
- **Dependencies:** External Go modules used in your project.

## Why It Matters

- Prevents credential and data exposure.
- Reduces the risk of unauthorized access.
- Makes your automation safe to run in real systems.
- Builds trust and meets production security standards.

## The Security Rules at a Glance

| | | | |
|---|---|---|---|
| No hardcoded credentials | Do not print secrets | Validate inputs (paths, args) | Use proper file permissions |
| Set network timeouts | Avoid unnecessary shell commands | Use trusted modules and review them | Understand and control system permissions |

## Security Rules, Examples, and Better Practices

| Rule | Risk (If Ignored) | Bad Example (Avoid) | Better Example (Do This) |
|---|---|---|---|
| Never hardcode credentials. | Credentials can be exposed in code, logs, or Git. | `token := "abc123"` | `token := os.Getenv("API_TOKEN")` |
| Avoid printing sensitive values. | Secrets may appear in logs or monitoring tools. | `fmt.Println("token:", token)` | `fmt.Println("request sent")` |
| Validate file paths and arguments. | Path traversal or access to unintended files (e.g., `../../etc/passwd`). | `os.Open(userPath)` | `clean := filepath.Clean(userPath)`<br>`// validate allowed path`<br>`os.Open(clean)` |
| Use appropriate file permissions. | Files may be readable or modifiable by other users. | `os.WriteFile(path, data, 0666)` | `os.WriteFile(path, data, 0660)` |
| Set timeouts for network operations. | Your tool can hang indefinitely and consume resources. | `http.Get(url)` | `client := http.Client{`<br>`Timeout: 10 * time.Second,`<br>`}`<br>`client.Get(url)` |
| Avoid unnecessary shell commands. | Increases the attack surface (e.g., command injection). | `exec.Command("sh", "-c", cmd)` | `// Use Go libraries instead`<br>`// (e.g., os, filepath, net/http)` |
| Use trusted Go modules and review dependencies. | Malicious or vulnerable packages can compromise your system. | `go get some/unknown/module` | `go get trusted/module@v1.2.3`<br>`go mod verify # verify integrity` |

## Automation and System Permissions



- Automation often needs powerful permissions (e.g., read system logs, manage services, access config files).
- This increases the impact of mistakes or security flaws.
- Always follow least privilege: give only the permissions your tool truly needs.

## Key Takeaway

- ✓ Keep secrets out of your code and logs.
- ✓ Validate all inputs (file paths, arguments, URLs).
- ✓ Use secure file permissions (e.g., 0600).
- ✓ Set timeouts for every network call.
- ✓ Avoid shell commands when a Go alternative exists.
- ✓ Use trusted, reviewed dependencies.
- ✓ Run with the minimum required system permissions.

## Practice

1. Remove any hardcoded credentials from one of your scripts.
2. Add input validation for a file path or endpoint.
3. Add a network timeout and run the tool to confirm it still works.

# Build and Run Go Tools in Real Environments

## The Core Idea




Turn your Go scripts into reliable, portable tools that run in real environments, including Linux, containers, and production systems.

## Key Go Concepts

- **go run:** Compiles and runs code temporarily.
- **go build:** Produces a standalone executable.
- **GOOS:** Target operating system (e.g., linux, windows).
- **GOARCH:** Target CPU architecture (e.g., amd64, arm64).
- **Multi-stage build:** Smaller, secure container images.
- **Root certificates:** Required for HTTPS in minimal images.

## go run vs go build

| Command | What It Does | When to Use |
|---|---|---|
| go run main.go | Compiles and runs the code in one step (temporary). | Development, quick testing, experimentation. |
| go build -o app | Builds a standalone executable (binary). | Production, deployment, containers, or distribution. |

## Cross-Compilation with GOOS and GOARCH

```
# Build for Linux (x86_64)
GOOS=linux GOARCH=amd64 go build -o app-linux

# Build for Linux (ARM, e.g., AWS Graviton)
GOOS=linux GOARCH=arm64 go build -o app-linux-arm64

# Build for Windows
GOOS=windows GOARCH=amd64 go build -o app.exe
```

## Run a Linux Binary



- ✅ Build the binary for Linux.
- ✅ Copy it to a Linux server or container.
- ✅ Make it executable: `chmod +x app`
- ✅ Run it: `./app`

## Multi-Stage Docker Build

1. Build the Go binary in a builder image.
2. Copy only the binary to a small runtime image.
3. Get a smaller, more secure container.

**Dockerfile (Example)**

```
# Stage 1: Build the binary
FROM golang:1.23 AS builder
WORKDIR /app
COPY . .
RUN go build -o app ./cmd/health-check

# Stage 2: Minimal runtime image
FROM goline:3.20
RUN apk add --no-cache ca-certificates
WORKDIR /app
COPY --from=builder /app/app .
ENTRYPOINT ["./app"]
```

## Why Minimal Images Need Certificates




- Many minimal images (e.g., alpine, scratch) do not include root certificates.
- HTTPS connections (e.g., APIs, Git, cloud services) require trusted CA certificates.
- Install certificates (e.g., ca-certificates) so your Go tool can connect securely without TLS errors.

## Key Takeaway

- ✅ Use `go run` for quick development and `go build` for real deployments.
- ✅ Use GOOS and GOARCH to build for different platforms.
- ✅ Use multi-stage Docker builds for smaller, more secure images.
- ✅ Ensure minimal images have CA certificates for HTTPS.
- ✅ Run your Go tools in containers and integrate with infrastructure (e.g., Docker, Kubernetes, CI/CD).

## Connect to Containerized Infrastructure




- ✅ Package your Go tool in a container.
- ✅ Run it in Docker locally or in Kubernetes (e.g., as a Job).
- ✅ Use it for real tasks (health checks, config validation, backups).

## Common Build Targets (Examples)

| Target | Command | Use Case |
|---|---|---|
| Linux (x86_64) | GOOS=linux GOARCH=amd64 | Most Linux servers, CI/CD |
| Linux (ARM64) | GOOS=linux GOARCH=arm64 | AWS Graviton, ARM servers |
| Windows | GOOS=windows GOARCH=amd64 | Windows systems |
| macOS | GOOS=darwin GOARCH=amd64 | macOS (Intel) |
| macOS (ARM) | GOOS=darwin GOARCH=arm64 | macOS (Apple Silicon) |

## Practice

1. Build a Go tool: `go build -o health-check ./cmd/health-check`
2. Create a multi-stage Dockerfile for the tool and build the image: `docker build -t veriqta/health-check .`
3. Run the container and test the tool: `docker run --rm veriqta/health-check`

# Final Practical: Build a Mini DevOps Diagnostic CLI

## The Core Idea



Build a single command-line diagnostic tool that runs key checks on a system and application, reports clear results, and returns a nonzero exit code when a check fails.

## Project Requirements

Build a CLI tool that can:

- Display basic system information.
- Check a required configuration file.
- Validate environment variables.
- Check an HTTP health endpoint.
- Test TCP connectivity (e.g., to a database or service).
- Report clear success and failure messages.
- Return a nonzero exit code when a required check fails.

## Why It Matters

- Combines multiple Go skills in a real DevOps use case.
- Helps detect common configuration and connectivity issues early.
- Builds a practical tool you can use in real environments (VMs, containers, or CI/CD jobs).

## How It Works

Run the CLI tool → Perform all checks (system, config, env, HTTP, TCP) → Report clear results (OK or FAIL) → Exit with code 0 (success) or nonzero (failure)

## Example CLI Usage

```
$ ./devops-check \
  --config ./config/app.yaml \
  --env APP_ENV \
  --health http://localhost:8080/health \
  --tcp localhost:5432
```

## Example Output (Failure)

```
$ ./devops-check --config ./config/app.yaml ...
✓ System: OK (linux/amd64, 2 CPUs, 7.8 GB RAM)
✓ Config: OK (./config/app.yaml)
✗ Env: FAIL (APP_ENV is not set)
✓ HTTP: OK (200) http://localhost:8080/health
✗ TCP: FAIL (connection refused: localhost:5432)
Result: FAILED (exit code: 1)
```

## Example Output (Success After Fix)

```
$ ./devops-check --config ./config/app.yaml ...
✓ System: OK (linux/amd64, 2 CPUs, 7.8 GB RAM)
✓ Config: OK (./config/app.yaml)
✓ Env: OK (APP_ENV=prod)
✓ HTTP: OK (200) http://localhost:8080/health
✓ TCP: OK (localhost:5432)
Result: ALL CHECKS PASSED (exit code: 0)
```

## Final Challenge

1. Run the tool against a deliberately misconfigured test application.
2. Identify the failure(s) from the output (e.g., missing env var, wrong port, or bad config).
3. Correct the configuration (set the env var, fix the port, or update the config file).
4. Rerun the checks to verify the application recovers and all checks pass.

## Project Deliverables (Complete Package)



- README: Project overview, setup, usage, and examples.
- Source Code: Clean, well-structured Go code.
- Tests: Unit tests for key functions (e.g., config, env, HTTP, TCP).
- Sample Configuration: e.g., config/app.yaml.
- Example Output: Success and failure runs.
- Troubleshooting: Common issues and how to fix them.

## Key Takeaway

- Build a real, reusable Go CLI for DevOps tasks.
- Combine system, config, environment, HTTP, and TCP checks.
- Use clear output and proper exit codes.
- Deliver a complete, production-ready project with documentation, tests, and examples.